

文章笔记

# 用 Codex 做数据分析与报告交付

五道口纳什 & Claude

2026-03-29

作者/来源: OpenAI

发布日期: 2025

文章链接: <https://developers.openai.com/codex/use-cases/datasets-and-reports>

# 目录

|                             |          |
|-----------------------------|----------|
| <b>1 引言：数据分析的核心目标</b>       | <b>2</b> |
| 1.1 本章小结                    | 2        |
| <b>2 定义分析问题</b>             | <b>2</b> |
| 2.1 运行示例：高速公路旁的房产估值         | 2        |
| 2.2 本章小结                    | 3        |
| <b>3 环境搭建与项目规范</b>          | <b>3</b> |
| 3.1 用 AGENTS.md 引导 Codex 行为 | 3        |
| 3.2 本章小结                    | 3        |
| <b>4 数据导入与初步审查</b>          | <b>3</b> |
| 4.1 本章小结                    | 4        |
| <b>5 数据整理与合并</b>            | <b>4</b> |
| 5.1 合并前的 Profiling          | 4        |
| 5.2 本章小结                    | 4        |
| <b>6 探索性分析与 Worktree 隔离</b> | <b>4</b> |
| 6.1 使用 Git Worktree 管理并行探索  | 5        |
| 6.2 本章小结                    | 5        |
| <b>7 建模</b>                 | <b>5</b> |
| 7.1 建模最佳实践                  | 5        |
| 7.2 本章小结                    | 5        |
| <b>8 结果沟通与交付</b>            | <b>6</b> |
| 8.1 本章小结                    | 6        |
| <b>9 Skills 与工具生态</b>       | <b>6</b> |
| 9.1 推荐技术栈                   | 7        |
| 9.2 本章小结                    | 7        |
| <b>10 总结与延伸</b>             | <b>7</b> |
| 10.1 拓展阅读                   | 7        |

# 1 引言：数据分析的核心目标

数据分析的本质是用**数据驱动决策**。目标不是分析本身，而是产生一个能帮助他人行动的 artifact（交付物）：为管理层提供的图表、为产品团队提供的实验报告、为研究者提供的模型评估、或指导日常运营的 dashboard。

## 数据分析循环框架

《R for Data Science》一书推广了一种经典的数据分析循环：

1. **Import**（导入）：加载原始数据
  2. **Tidy**（整理）：清洗、规范化数据格式
  3. **Transform / Visualize / Model**（变换 / 可视化 / 建模）：在三者之间迭代，逐步建立对数据的理解
  4. **Communicate**（沟通）：将结果包装为他人可消费的形式
- 编程贯穿整个循环。Codex 的价值在于加速你在循环中的每一步。

## Codex 的定位

Codex 不是用来生成一次性 notebook 的工具。它的目标是帮助你构建一个**其他人可以审查、信任并重新运行**的完整工作流（reproducible workflow）。

## 1.1 本章小结

数据分析应当以“产出可行动的交付物”为导向，Codex 帮助你在 import-tidy-transform-visualize-model-communicate 循环中更高效地推进。

# 2 定义分析问题

在动手之前，首先要选定一个**具体的问题**。问题越明确，Codex 就越能理解你的目标并提供针对性帮助。

## 2.1 运行示例：高速公路旁的房产估值

文章使用的贯穿示例是：靠近高速公路的房屋是否在估值上偏低？

假设有两个数据集：一个包含房产价值或成交价，另一个包含位置、地块或高速公路邻近度信息。真正的工作不只是跑一个模型，而是：

1. 使输入数据可信（data quality）
2. 记录合并逻辑（document the joins）
3. 压力测试结果（pressure-test the result）
4. 最终产生一个别人能直接使用的 artifact

## 常见误区：问题定义过于模糊

“分析一下这个数据集”这样的指令对 Codex 几乎没有帮助。你需要给出一个具体的、可验证的问题（如“高速公路 500m 内的房屋均价是否低于 1km 外的房屋”），Codex 才能有效拆分子任务。

## 2.2 本章小结

明确、具体的分析问题是整个工作流的起点。越精确的问题，Codex 越能有效拆分为子步骤并给出针对性帮助。

## 3 环境搭建与项目规范

开始新的数据分析项目时，需要先设置好三个方面：

- **Environment** (环境): Python 环境、包管理器、目录结构、输出约定
- **Skills** (技能): 把重复工作流 (如 notebook 清理、spreadsheet 导出、最终报告打包) 封装为可复用的 skill
- **Worktrees** (工作树): 将不同探索方向隔离到不同 worktree, 避免假设、合并策略或可视化分支之间互相干扰

### 3.1 用 AGENTS.md 引导 Codex 行为

在接触数据之前，先告诉 Codex 项目的行为规范。个人默认设置放在 `~/.codex/AGENTS.md`, 项目级规则放在仓库根目录的 `AGENTS.md`。

```
1 ## Data analysis defaults
2 - Use `uv run` or the project's existing Python environment.
3 - Keep source data in `data/raw/` and write cleaned data
4   to `data/processed/`.
5 - Put exploratory notebooks in `analysis/` and final
6   artifacts in `output/`.
7 - Never overwrite raw files.
8 - Prefer scripts or checked-in notebooks over unnamed
9   scratch cells.
10 - Before merging datasets, report candidate keys, null
11   rates, and join coverage.
```

Listing 1: 一个简洁的 AGENTS.md 示例

#### AGENTS.md 的作用

AGENTS.md 是 Codex 的“项目宪法”。它确保 Codex 在每次交互中都遵守统一的目录结构、数据保护规则和工作流约定，而不需要你在每条 prompt 里重复说明。

### 3.2 本章小结

项目规范先行：通过 AGENTS.md 定义环境、目录结构和行为约束，通过 Skills 封装重复流程，通过 Worktrees 隔离并行探索。

## 4 数据导入与初步审查

最快的起步方式是把文件路径贴给 Codex，让它做初步检查。此阶段需要回答的关键问题：

- 有哪些文件格式？

- 每个数据集大致代表什么？
- 哪些列可能是目标变量、标识符、日期、位置或度量？
- 明显的`数据质量问题`在哪里？

### 不要过早下结论

此阶段只做`数据盘点` (inventory) 和`解释说明` (explanation)，不要急于得出分析结论。先理解数据的“长相”，再进入清洗和合并。

## 4.1 本章小结

数据导入阶段的核心是“看清数据”而非“分析数据”。让 Codex 做好文件格式识别、字段解释和质量初筛。

# 5 数据整理与合并

大多数真正的工作从这里开始。你通常面对两个或更多数据集，主键不明确，朴素的 merge 可能丢数据或产生重复行。

## 5.1 合并前的 Profiling

要求 Codex 在执行合并之前先做以下检查：

1. `检查候选键的唯一性` (uniqueness of candidate keys)
2. `度量空值率和格式差异` (null rates and formatting differences)
3. `规范化明显的格式问题`：大小写、空格、地址格式等
4. `执行试验性 join 并报告匹配率` (trial joins and match rates)
5. `推荐最安全的合并策略`，然后才写入最终合并文件

### “先 Profile，再 Merge”原则

如果需要构造最优的 join key (如规范化地址、从多列拼接的地块标识符、或基于位置的 join)，让 Codex 先解释 trade-off 和边界情况，**你确认后再执行合并**。这是防止数据泄漏和合并错误的`关键步骤`。

## 5.2 本章小结

数据合并是分析中最容易出错的环节。核心原则是“先审查、再合并”：让 Codex 报告候选键、空值率、匹配率和策略建议，确认无误后才执行。

# 6 探索性分析与 Worktree 隔离

探索性数据分析 (EDA) 是 Codex 最受益于干净隔离的阶段。一个 worktree 可以测试地址清洗或特征工程，另一个专注于图表或替代建模方向。

## 6.1 使用 Git Worktree 管理并行探索

Codex App 内置了 worktree 支持。在终端中也可以直接使用 Git worktree:

```
1 git worktree add ../analysis-highway-eda \  
2   -b analysis/highway-eda  
3 git worktree add ../analysis-model-comparison \  
4   -b analysis/highway-modeling
```

Listing 2: 创建 worktree 隔离不同探索方向

### 为什么要用 Worktree ?

Worktree 的价值在于:

- 每个 diff 可独立 review, 不会混入无关改动
- 防止一个长线程中混入不兼容的想法
- 保持每条探索路径的可回溯性

在贯穿示例中, 这一步要做的是: 比较靠近高速公路的房屋与远离高速公路的房屋, 检查异常值, 审视缺失值模式, 判断观察到的效应是真实的还是反映了社区构成、房屋面积或其他混杂因素。

## 6.2 本章小结

EDA 阶段利用 Worktree 将不同假设和探索方向物理隔离, 保持每个分支的 diff 清晰可审查。

## 7 建模

并非每个分析都需要复杂模型。从**可解释的基线** (interpretable baseline) 开始。

### 7.1 建模最佳实践

对于高速公路房产问题, 合理的第一步是回归或其他透明模型, 在控制面积、房龄、位置等因素后, 估计高速公路邻近度与房产价值之间的关系。

要求 Codex 明确以下要素:

1. **目标变量和特征定义** (target variable and feature definitions)
2. **纳入哪些控制变量及原因** (which controls and why)
3. **数据泄漏风险和排除项** (leakage risks and exclusions)
4. **划分方式、评估方法和不确定性估计** (split, evaluation, uncertainty)
5. **用通俗语言解释结果含义** (plain language interpretation)

### 弱模型也有价值

如果第一个模型效果不佳, 这本身就是有用的信号。它能告诉你问题出在模型选择、特征工程、数据合并质量还是问题定义本身。不要因为结果不好就直接跳到复杂模型。

### 7.2 本章小结

建模应从可解释的基线出发, 要求 Codex 明确目标、特征、控制变量、泄漏风险和结果解读。弱模型是诊断问题的重要信号。

## 8 结果沟通与交付

分析只有在他人能消费时才有用。根据受众选择合适的交付格式：

| 交付形式              | 适用场景          |
|-------------------|---------------|
| Markdown memo     | 技术协作者         |
| CSV / Spreadsheet | 下游运营团队        |
| .docx 报告          | 需要排版和表格的正式文档  |
| PDF               | 最终交付物或附录      |
| 静态报告站点            | 需要以 URL 分享的结果 |

表 1: 不同受众对应的交付格式

### 别忘了写 Caveats

如果 join 质量不完美、存在采样偏差、或模型假设脆弱，Codex 应当在交付物中**明确说明这些局限**。诚实的 caveats 是建立信任的关键。

### 8.1 本章小结

交付物的形式应匹配受众需求。无论选择哪种格式，都必须包含诚实的局限性说明。

## 9 Skills 与工具生态

Codex 提供了一系列与数据分析工作流高度契合的 Skills（技能插件）：

| Skill              | 用途                        |
|--------------------|---------------------------|
| \$spreadsheet      | CSV、TSV、Excel 的查看与编辑导出    |
| \$jupyter-notebook | 当交付物需要保持 notebook 原生格式时使用 |
| \$doc / \$pdf      | 面向利益相关者的正式输出              |
| \$vercel-deploy    | 将结果以 URL 形式分享             |

表 2: Codex 数据分析相关 Skills

### 从临时 Prompt 到持久 Skill

当工作流稳定后，应将重复性步骤（如 `refresh-data`、`merge-and-qa`、`publish-weekly-report`）封装为仓库本地的 Skill。这比每次都粘贴同样的流程 prompt 要高效得多。

## 9.1 推荐技术栈

| 需求  | 默认选择                        | 原因                         |
|-----|-----------------------------|----------------------------|
| 分析栈 | pandas + matplotlib/seaborn | 导入、profiling、join、清洗、图表一站式 |
| 建模  | statsmodels / scikit-learn  | 先可解释基线，再考虑复杂模型             |

表 3: 推荐技术栈

## 9.2 本章小结

利用 Codex 内置 Skills 加速交付， workflows 稳定后将重复步骤封装为仓库级 Skill。技术栈以 pandas + statsmodels/sklearn 为核心。

# 10 总结与延伸

本文梳理了使用 OpenAI Codex 进行数据分析与报告交付的完整工作流，核心要点如下：

1. **问题先行**：先确定一个具体、可验证的分析问题，再动手写代码
2. **规范先行**：通过 AGENTS.md 定义项目规范，让 Codex 在一致的约束下工作
3. **数据先行**：先盘点数据 (inventory)，再清洗合并，“先 Profile 再 Merge”
4. **隔离探索**：用 Worktree 将不同方向的 EDA 物理隔离，保持 diff 可审查
5. **基线优先**：从可解释模型开始，弱结果本身也是诊断信号
6. **交付导向**：最终输出的不是 notebook，而是受众可直接使用的 artifact
7. **技能复用**：将稳定的工作流封装为 Skill，提升长期效率

## 10.1 拓展阅读

- OpenAI Codex 官方文档: <https://developers.openai.com/codex/>
- Codex Skills 文档: <https://developers.openai.com/codex/skills/>
- 《R for Data Science》数据分析循环框架: <https://r4ds.had.co.nz/>
- Codex Worktree 使用指南: <https://developers.openai.com/codex/worktrees/>